

Intelligent Network Intrusion Detection Using Machine Learning and Deep Learning

[Dana Alghamdi, Rana Alziyadi, Raneem Alqahtani, Essa Almutawa]

Data Science Bootcamp

September 2026

Abstract

Network intrusion detection is complicated by severe class imbalance: benign flows vastly outnumber attacks, and a classifier that predicts the majority class achieves high accuracy while detecting nothing. This study develops and evaluates a binary intrusion detection system on the CIC-IDS2017 dataset, comparing three approaches that represent distinct learning paradigms: a class-weighted Logistic Regression baseline, a LightGBM gradient-boosting ensemble, and an unsupervised Autoencoder trained only on benign traffic. Repeated flow patterns were collapsed before splitting and all feature selection was fitted on the training set, so that no test record repeats a training pattern. Thresholds were tuned on a separate validation set, and Average Precision (AP) served as the primary metric. LightGBM performed best (AP 0.9999, recall 0.9983), detecting 84,893 of 85,036 test attacks with 165 false alarms across 419,158 benign flows. A held-out attack-family experiment, in which one attack family was withheld from classifier training, showed LightGBM's recall on that family falling from 99.91% to 10.04%, while the Autoencoder, trained on benign traffic only, detected 61.08% at its F1-tuned threshold and 99.01% at a threshold set without any attack labels, at a false-positive rate of 5.06%. The results show that near-perfect supervised performance on known attacks does not transfer to unseen ones, which motivates pairing a supervised detector with an unsupervised one.

Keywords: intrusion detection; class imbalance; gradient boosting; anomaly detection; autoencoder; CIC-IDS2017

1. Introduction

Network intrusion detection systems (NIDS) play an important role in protecting computer networks by identifying malicious or suspicious activity. As network environments become more complex and attack techniques continue to evolve, conventional rule-based detection may be limited when traffic patterns change or when attacks do not match previously defined signatures. For this reason, machine-learning and deep-learning approaches have increasingly been investigated as methods for learning behavioral patterns directly from network traffic [2], [3].

One of the main challenges in network intrusion detection is class imbalance. Benign traffic typically represents the majority of network activity, while malicious traffic — and particularly some individual attack families — may occur much less frequently. Under these conditions, overall accuracy can provide an incomplete view of performance because a classifier may correctly identify most benign traffic while still failing to detect important attacks [1]. A reliable evaluation should therefore consider attack detection, false alarms, and metrics that remain informative under unequal class distributions.

This study investigates binary network intrusion detection using the CIC-IDS2017 dataset. Three approaches representing different learning strategies are evaluated: weighted Logistic Regression as a linear supervised baseline, LightGBM as a non-linear gradient-boosting classifier, and an Autoencoder trained only on benign traffic as an anomaly-detection model. Comparing these approaches provides insight into how different modeling assumptions affect intrusion-detection performance.

The study also emphasizes evaluation reliability. Data leakage and repeated network-flow patterns are explicitly addressed, feature selection and preprocessing are learned from the training data only, and separate training, validation, and test sets are used. Performance is assessed using precision, recall, F1-score, false-positive rate and AP, with AP treated as the primary metric. In addition to aggregate binary performance, per-attack-family recall and a held-out attack experiment are used to examine model behavior that may not be visible from overall results alone.

2. Related Work

2.1 Class Imbalance in Intrusion Detection

Class imbalance is a recurring challenge in network intrusion detection because benign traffic generally occurs more frequently than malicious traffic. This imbalance can cause classification algorithms to favor the majority class and reduce their ability to detect less frequent attacks [1].

Previous studies have addressed this problem through techniques such as oversampling, undersampling, hybrid resampling, class weighting, and imbalance-aware evaluation metrics. Ahsan et al. [1], for example, examined strategies for handling imbalanced intrusion-detection data and emphasized the importance of evaluating minority-class detection and false alarms rather than relying on accuracy alone.

In this study, imbalance is handled primarily through class weighting in the supervised models and through evaluation metrics that better reflect attack-detection performance. AP, recall, F1-score and FPR are therefore considered instead of overall accuracy.

2.2 Supervised Approaches

Machine-learning and deep-learning methods have been widely investigated for network intrusion detection. Ahmad et al. [3] reviewed a broad range of ML- and DL-based NIDS approaches and identified persistent challenges including false alarms, class imbalance, and generalization to previously unseen attacks.

More recent work has explored increasingly complex supervised architectures. Hu et al. [5], for example, evaluated a Self-Attention and BiGRU-based model on CIC-IDS2017 together with techniques intended to improve minority-class detection. Such work also highlights that strong aggregate performance does not necessarily imply equally strong performance across individual attack categories.

These observations motivate the supervised comparison used in this study. Logistic Regression provides a linear baseline, while LightGBM represents a stronger non-linear model capable of learning complex interactions among network-flow features. Their performance is evaluated both globally and at the individual attack-family level.

2.3 Anomaly-Based and Deep Learning Approaches

Anomaly-based intrusion detection follows a different principle from conventional supervised classification. Rather than learning only the boundaries between known benign and malicious examples, anomaly-based methods attempt to model normal behavior and identify observations that deviate from it.

Deep-learning approaches, including Autoencoders, have been widely studied for this purpose [2]. Aldweesh et al. [2] reviewed deep-learning methods for anomaly-based intrusion detection and identified challenges related to evaluation methodology, dataset quality, deployment, and the detection of previously unseen attacks.

The Autoencoder in this study is trained exclusively on benign traffic and uses reconstruction error as an anomaly score. Because it does not learn attack-specific patterns during training, it provides a complementary comparison with supervised models, particularly when evaluating an attack family that is deliberately excluded from supervised training.

2.4 The CIC-IDS2017 Benchmark

CIC-IDS2017 was developed as a benchmark for evaluating intrusion-detection systems using labeled benign and malicious network traffic [4]. It contains multiple attack scenarios, including DoS, DDoS, Brute Force, PortScan, Botnet, Web Attacks, Infiltration, and Heartbleed, together with normal user activity.

The dataset has also been widely used in machine-learning and deep-learning intrusion-detection research, including recent supervised studies such as [5]. This provides a useful basis for comparison with previous work. At the same time, CIC-IDS2017 contains substantial differences in the number of observations available for individual attack families, making it suitable for studying both aggregate performance and the detection of rare attacks.

3. Dataset

This study uses the CIC-IDS2017 dataset for binary network intrusion detection [4]. The dataset is distributed as eight daily CSV files containing 2,830,743 network-flow records in total, with 78 engineered flow features describing characteristics of network communication, plus the label column. A ninth column recording the source capture file was added when the eight files were merged, giving 80 columns in the working dataset. The original multiclass labels were retained

separately for later per-attack-family analysis, while the modeling task was converted into binary classification: BENIGN was assigned to the negative class (0) and every other label to the positive ATTACK class (1).

The original binary distribution is clearly imbalanced. Benign traffic accounts for 2,273,097 flows (80.30%), while attack traffic accounts for 557,646 flows (19.70%), corresponding to approximately one attack flow for every 4.1 benign flows. However, this aggregate ratio does not fully describe the imbalance present within individual attack families.

Table 1. Distribution of traffic classes in CIC-IDS2017.

Traffic class	Flows	Share of dataset
BENIGN	2,273,097	80.30%
DoS Hulk	231,073	8.16%
PortScan	158,930	5.61%
DDoS	128,027	4.52%
DoS GoldenEye	10,293	0.36%
FTP-Patator	7,938	0.28%
SSH-Patator	5,897	0.21%
DoS slowloris	5,796	0.20%
DoS Slowhttptest	5,499	0.19%
Bot	1,966	0.07%
Web Attack – Brute Force	1,507	0.05%
Web Attack – XSS	652	0.02%
Infiltration	36	0.001%
Web Attack – SQL Injection	21	0.001%
Heartbleed	11	0.0004%

The class distribution varies considerably across attack categories. DoS Hulk, PortScan, and DDoS contain more than one hundred thousand records each, whereas several attack families contain only a small number of observations. Heartbleed contains only 11 flows, SQL Injection contains 21, and Infiltration contains 36. Consequently, high overall binary performance may still conceal poor detection of rare attack families.

For this reason, the original attack-family labels were preserved separately for evaluation but were never included as model input features. This allows performance to be examined both at the overall binary level and individually for each attack family.

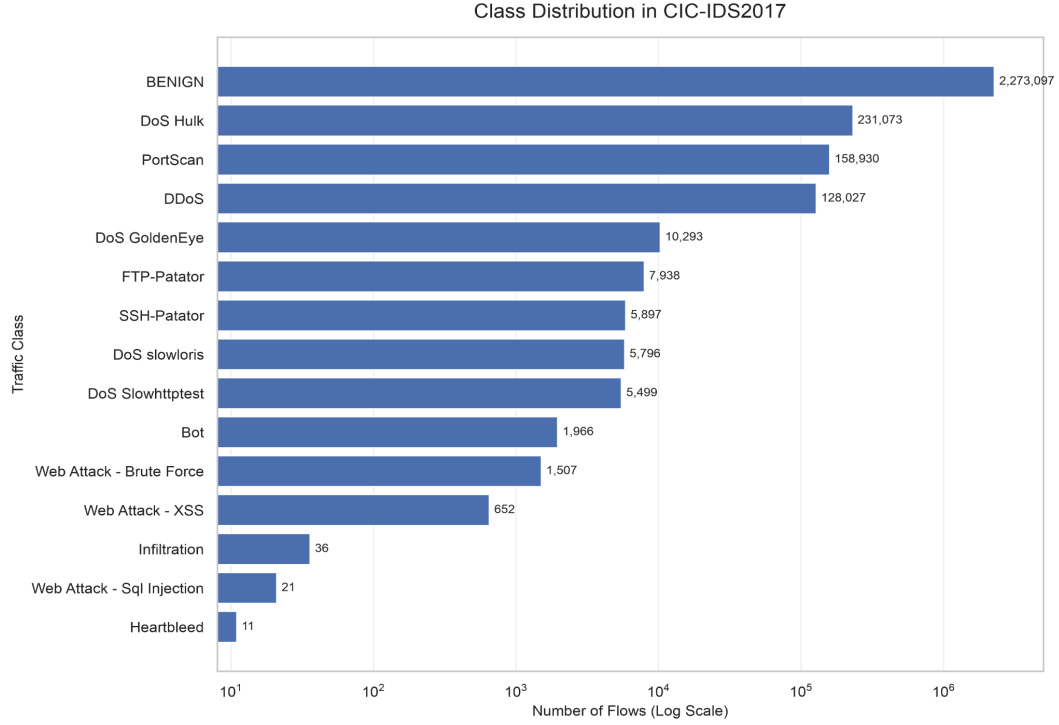


Figure 1. Class distribution in CIC-IDS2017: benign traffic and the fourteen attack families, logarithmic scale. .

4. Methodology

The methodology was designed to provide a reproducible evaluation while minimizing information leakage between training and held-out data. The modeling pipeline operated on the cleaned CIC-IDS2017 dataset produced in the exploratory stage, containing 2,830,743 records. A strict separation was maintained between operations that can be performed independently of the data distribution and operations that learn information from the data. Feature selection, imputation, transformation, and model-related decisions were therefore fitted using the training or validation data only and were not influenced by the final test set.

4.1 Data Preparation and Feature Selection

Because CIC-IDS2017 already provides engineered network-flow features, the preparation stage focused on data quality, leakage prevention, duplicate handling, and feature reduction rather than additional feature construction.

Column names were standardized before further processing. Some rate-based features can produce infinite values when the flow duration is zero; these values were converted to missing values for later imputation. After this conversion, and after physically impossible negative values

were set to missing during data preparation, 9,314 missing cells were present across 5,792 rows, representing approximately 0.2046% of the original records.

Potential leakage sources were then excluded from the feature matrix. The pipeline checked `Label`, `Label_binary` and `source_file` as variables that should never be used as model inputs. In the final working dataset, `Label` and `source_file` were present and removed, leaving 78 numerical flow features. `Label` directly represents the prediction target, while `source_file` identifies the capture file or day. Because several capture days contain only one attack scenario, retaining this metadata could allow a model to infer the class from the collection context rather than from network behavior.

Exact duplicate flow patterns were also addressed before data splitting. This was necessary because repeated flows could otherwise be randomly distributed across both training and test sets, causing part of the final score to reflect recognition of previously observed feature patterns rather than generalization.

Repeated patterns associated with the same label were collapsed so that only one representative remained. When identical feature vectors appeared with contradictory labels, the entire pattern was removed, because no model can distinguish observations with exactly the same inputs but different targets. This procedure removed 302,757 repeated copies and an additional 7,020 rows belonging to 698 contradictory patterns. The dataset was therefore reduced from 2,830,743 records to 2,520,966 distinct flow patterns. Because duplicate records were not distributed equally across classes, the overall attack rate decreased from 19.70% to 16.87%.

Feature reduction was subsequently fitted on the training set only. First, eight zero-variance features were removed because they provided no discriminatory information. Highly redundant features were then identified using the absolute Pearson correlation coefficient. Correlations were calculated from a reproducible sample of up to 100,000 training observations, and one feature from each pair with an absolute correlation greater than 0.95 was removed. This eliminated 26 additional features and produced a final input space of 44 features. The selected feature list was then applied unchanged to the validation and test sets.

Table 2. Feature reduction pipeline.

Step	Features remaining	Removed	Rationale
Original columns	80	—	Input columns before modeling

Step	Features remaining	Removed	Rationale
Remove leakage sources	78	2	Label and capture metadata
Remove constant features	70	8	Zero variance in training set
Remove redundant features	44	26	Correlation above 0.95 in training set

The reduction process therefore addressed three distinct issues: target leakage, non-informative constant variables, and strong redundancy among correlated features. The latter two decisions were determined from the training data only, so that held-out observations did not influence the final feature representation.

4.2 Data Splitting

After duplicate and contradictory patterns were removed, the remaining 2,520,966 records were divided using a stratified three-way split. Sixty percent of the data was assigned to training, 20% to validation and 20% to testing, giving 1,512,579 training records, 504,193 validation records and 504,194 test records.

Three separate sets were used because model development involved decisions based on observed performance. The training set was used to estimate model parameters and to fit all data-dependent preprocessing steps. The validation set was used for decision-threshold selection and LightGBM early stopping. The test set was reserved exclusively for the final reported evaluation. This separation prevents decisions such as threshold selection from being influenced by the test set and therefore reduces optimistic bias in the final performance estimates.

An additional independence check was performed after splitting. Exact feature patterns from the test set were compared with those observed during training, and the pipeline reported zero test rows repeating a training pattern. This provided a direct check that exact duplicate leakage had been removed before final evaluation.

4.3 Models

Three models were evaluated, each representing a different learning approach: a linear supervised baseline, a non-linear tree-based model, and an unsupervised deep-learning model.

Logistic Regression — linear supervised baseline. Logistic Regression was used to evaluate how effectively a linear decision boundary could distinguish benign from attack traffic. Class imbalance was handled using `class_weight='balanced'`, which gives greater importance

to the minority class during training. The model used the lbfgs solver with `max_iter=1000`, raised from the scikit-learn default of 100 so that the optimiser would not be stopped before convergence; it converged after 220 iterations, so the limit was never reached.

LightGBM — gradient-boosted decision trees. LightGBM was used as the non-linear supervised model because gradient-boosted trees can capture complex relationships and interactions between network-flow features. The model was configured with a learning rate of 0.05, 63 leaves, and a minimum of 50 samples per leaf. Class imbalance was addressed using `scale_pos_weight`, calculated from the benign-to-attack ratio in the training data. Subsampling, feature sampling and L1/L2 regularization were included to control model complexity. Training allowed up to 1,000 boosting rounds, with early stopping after 50 rounds without improvement in validation average precision. Key parameters: `n_estimators=1000`, `learning_rate=0.05`, `num_leaves=63`, `min_child_samples=50`, `subsample=0.8`, `subsample_freq=1`, `colsample_bytree=0.8`, `reg_alpha=0.1`, `reg_lambda=0.1`.

Autoencoder — unsupervised deep-learning model. The Autoencoder was used as an anomaly-detection model and was trained exclusively on benign traffic rather than using attack labels. After feature reduction, 44 input features were used. Its hourglass architecture was 44-64-32-16-32-64-44, with a 16-neuron bottleneck that forces the network to learn a compressed representation of normal traffic behavior. ReLU activation was used in the hidden layers and sigmoid activation in the output layer, matching the bounded range of the transformed inputs. Batch normalization and a dropout rate of 0.1 were included to improve training stability. The model was optimized using Adam with a learning rate of 0.001 and mean squared error as the reconstruction loss, for a maximum of 30 epochs with a batch size of 512 and a 10% validation split. Early stopping with a patience of 5 epochs and learning-rate reduction with a patience of 3 epochs were used during training. After training, the mean reconstruction error for each flow was used as the anomaly score, with larger errors indicating greater deviation from learned benign behavior.

4.4 Preprocessing

Two preprocessing tracks were used because the evaluated models have different numerical requirements. LightGBM is tree-based and makes decisions using feature thresholds, so monotonic feature scaling does not materially affect its splitting behavior. Logistic Regression

and the Autoencoder, in contrast, are scale-sensitive and therefore require a more controlled numerical representation.

For LightGBM, missing values were replaced using median imputation without additional scaling. The median was preferred to the mean because many CIC-IDS2017 variables are highly skewed and contain extreme observations. The imputer was fitted using the training set and then applied unchanged to validation and test data.

For Logistic Regression and the Autoencoder, the same median imputation was followed by a QuantileTransformer. The transformer used 1,000 quantiles, a maximum subsample of 200,000 training observations, a fixed random state of 42, and a uniform output distribution.

A quantile-based transformation was selected because conventional standardization rescales a distribution without correcting severe skewness. This is particularly relevant for the Autoencoder, whose squared reconstruction-error objective can be strongly influenced by extreme values. A normal output distribution was also considered but was not used because several features are heavily zero-inflated, and repeated zeros can be mapped near the extreme boundary of a normal quantile transformation. The uniform transformation instead provides a bounded range between approximately 0 and 1.

All imputation and transformation parameters were learned exclusively from the training data and then reused without refitting on the validation and test sets, preventing held-out information from influencing preprocessing decisions.

5. Results

This section reports the performance of each model on the held-out test set. Their interpretation is given in Section 6.

5.1 Overall Performance

All three models were evaluated on the same held-out test set of 504,194 flows, containing 85,036 attacks and 419,158 benign flows. Each model was applied at its own decision threshold, selected on the validation set. AP summarises the precision–recall curve as a single value and is computed using scikit-learn's `average_precision_score`.

Table 3. Model performance on the held-out test set, each at its validation-tuned threshold.

Model	Threshold	Precision	Recall	F1	AP	FPR
Logistic Regression	0.7135	0.9256	0.9621	0.9435	0.9308	0.0157
LightGBM	0.8467	0.9981	0.9983	0.9982	0.9999*	0.0004
Autoencoder	0.0020	0.8291	0.7414	0.7828	0.8511	0.0310

* 0.999939; not exactly 1.

Table 4. The same results expressed as raw counts.

Model	Attacks detected	Attacks missed	False alarms
Logistic Regression	81,811	3,225	6,573
LightGBM	84,893	143	165
Autoencoder	63,046	21,990	12,998

The Autoencoder is additionally reported at a second threshold, set from benign validation traffic alone and therefore requiring no attack labels: the 95th percentile of benign reconstruction error, 0.0015. At that threshold it detects 67,110 attacks, misses 17,926, and raises 21,226 false alarms.

5.2 Per-Family Detection

The original attack-family labels were recovered for the test set only, and recall was computed separately for each family. Family labels were never used as model inputs.

Table 5. Recall by attack family on the test set (%).

Attack family	Test records	Log. Regression	LightGBM	Autoencoder
DoS Hulk	34,635	96.31	99.96	89.97
DDoS	25,632	99.84	99.97	78.86
PortScan	18,092	98.46	99.99	49.79
DoS GoldenEye	1,975	78.78	100.00	21.16
FTP-Patator	1,115	99.82	99.91	61.08
DoS slowloris	1,103	94.38	99.82	69.36
DoS Slowhttptest	990	89.80	100.00	42.22
SSH-Patator	635	13.70	99.69	57.01
Bot	399	0.00	72.68	3.26
Web Attack – Brute Force	297	79.46	98.99	0.00
Web Attack – XSS	149	84.56	100.00	0.00
Infiltration	7	0.00	57.14	85.71
Web Attack – SQL Injection	6	16.67	83.33	0.00

Attack family	Test records	Log. Regression	LightGBM	Autoencoder
Heartbleed	1	0.00	100.00	100.00

The three families at the foot of the table hold fewer than ten test records each; this is addressed as a limitation in Section 7.

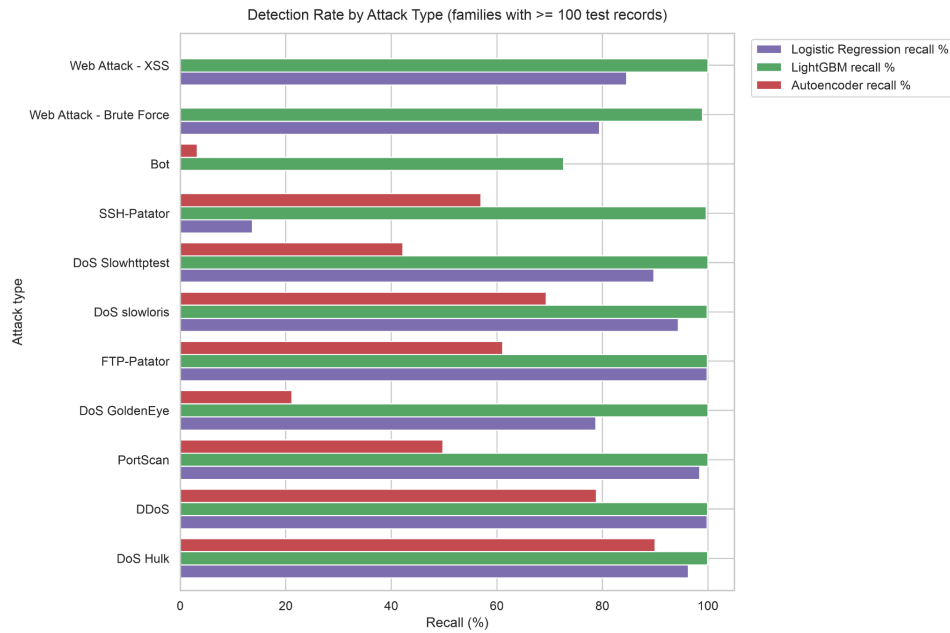


Figure 2. Detection rate by attack family for the three models, restricted to families with at least 100 test records.

5.3 Threshold Sensitivity

The decision threshold of the selected model was swept from 0.05 to 0.95 in nineteen equal steps on the validation set, recording precision, recall and F1 together with the resulting counts of missed attacks and false alarms. Six representative points are shown in Table 6.

Table 6. LightGBM across the threshold range, on the validation set.

Threshold	Precision	Recall	F1	Attacks missed	False alarms
0.05	0.9948	0.9999	0.9973	8	446
0.25	0.9964	0.9998	0.9981	13	307
0.50	0.9971	0.9996	0.9984	32	248
0.75	0.9979	0.9990	0.9985	81	177
0.85	0.9984	0.9987	0.9986	112	133
0.95	0.9988	0.9979	0.9983	177	106

Across the full range F1 varies by 0.0013, from 0.9973 to 0.9986. Over the same range the number of missed attacks rises from 8 to 177 while the number of false alarms falls from 446 to 106.

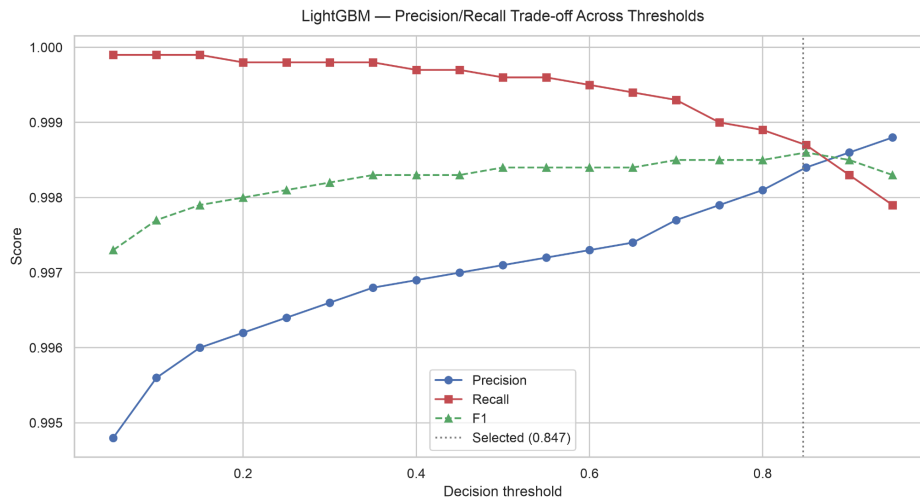


Figure 3. LightGBM precision, recall and F1 as a function of the decision threshold, on the validation set. The dotted line marks the selected threshold of 0.847.

5.4 Held-Out Attack-Family Experiment

One attack family, FTP-Patator, was removed from classifier training entirely, and also from the validation data used for early stopping, so that it could not be learned indirectly. This removed 3,672 training rows, leaving 1,508,907. Both supervised models were retrained on the reduced set. The Autoencoder required no retraining, since it never uses attack labels; only its threshold was affected.

Each model is reported at more than one threshold, because a threshold chosen on data containing the family is itself a form of exposure to it. “Transferred” is the original validation threshold; “family-free” is re-selected on validation data with the family removed; the Autoencoder is additionally reported at its label-free benign-percentile threshold.

Evaluation was performed on 420,273 flows: the 1,115 FTP-Patator records in the test set together with all 419,158 benign test flows.

Table 7. Detection of FTP-Patator with and without the family present in training.

Model	Threshold source	Family in training	Recall	False alarms
Logistic Regression	original validation F1	Yes	99.82%	6,573
Logistic Regression	transferred	No	99.28%	6,471
Logistic Regression	family-free validation F1	No	99.28%	6,343
LightGBM	original validation F1	Yes	99.91%	165
LightGBM	transferred	No	10.04%	151
LightGBM	family-free validation F1	No	2.42%	137
Autoencoder	original validation F1	Never	61.08%	12,998
Autoencoder	family-free validation F1	Never	52.91%	12,420
Autoencoder	benign validation P95 (no labels)	Never	99.01%	21,226

5.5 Validity Checks

Three checks were run before the results above were reported.

Overfitting. Each model was scored on its own training data and the result compared with the test figures. The train-minus-test AP gaps were -0.0005 for Logistic Regression, 0.0000 for LightGBM and -0.0010 for the Autoencoder.

Feature-level leakage. Each feature was evaluated on its own as a single-feature classifier, on a sample of 200,000 training rows. The most discriminative feature, Fwd IAT Std, reached a ROC-AUC of 0.7472.

Metric suitability. A trivial classifier labelling every flow benign was evaluated on the test set. It reached 83.13% accuracy while detecting none of the 85,036 attacks.

6. Discussion

Section 5 reports what each model achieved. This section explains why the models differ, where the aggregate figures conceal a weakness, and how far the results can be trusted.

6.1 The Detection Problem Is Fundamentally Non-Linear

The difference between the two supervised models in Table 3 measures how much this problem depends on combinations of features rather than on any feature considered alone. A linear model draws one boundary through the feature space; a gradient-boosted ensemble can carve out regions defined by several features at once.

The clearest evidence sits inside one attack category. Table 5 reports two brute-force families of comparable size. On FTP-Patator the two models are effectively equivalent. On SSH-Patator the baseline detects 13.70% against LightGBM's 99.69% — an 86-point gap between two attacks of the same kind. Sample size cannot explain it.

A plausible explanation, not tested here, is encryption. SSH sessions carry encrypted payloads, so the packet-length and timing statistics that distinguish a brute-force attempt from ordinary traffic become weak individually. Detection then requires combining several weak indicators, which one linear boundary cannot express. The practical consequence is that the baseline's aggregate recall of 0.9621 is not evenly distributed: it is carried by the large flood families and collapses on families whose signal is spread across features.

6.2 Bot Traffic Is the Limiting Factor

Bot is the final model's weakest family in Table 5, at 72.68%. Its importance is larger than that figure suggests: of the 143 attacks LightGBM misses in total, roughly 109 — about three quarters — are Bot flows.

This reframes the limitation. The model's remaining errors are not spread thinly across the dataset, to be recovered by tuning; they are concentrated in one traffic type. No change to the decision threshold reaches them, and improving Bot detection would move the headline numbers further than any amount of threshold optimization.

Scarcity is not the cause. Bot contributes three times as many flows as the XSS web-attack family (Table 1), on which the model reaches 100%. The cause is what the traffic is. Bot flows are the communication between an infected machine and the server controlling it, and that channel is built to be unremarkable: short, periodic connections that look like routine web browsing once reduced to flow statistics.

Neither alternative compensates: Logistic Regression detects none of the 399 Bot flows in the test set and the Autoencoder 3.26%, so this is a gap in the study as a whole rather than a weakness of one model.

Bot is also the only attack family in the capture without a dominant destination port: its flows are spread across 706 ports, with 8080 covering only 64%. PortScan shares this property yet is detected almost perfectly, so the absence of a port shortcut is not sufficient on its own to explain

poor detection — but Bot lacks both that shortcut and a distinctive flow signature, which is consistent with it being the family every model here handles worst.

6.3 The Unsupervised Model Is Selective, Not Simply Weak

The Autoencoder scores a flow by how badly it reconstructs it, so it detects whatever is statistically unusual relative to normal traffic. The natural expectation is that this means high-volume flood attacks and nothing else. Table 5 shows a more specific pattern.

DoS GoldenEye is the case that separates the two readings. It is a denial-of-service attack, yet the Autoencoder detects 21.16% of it against 89.97% for DoS Hulk — a gap of nearly seventy points inside one attack category. GoldenEye issues HTTP requests that individually resemble legitimate browsing, so what makes it an attack is the intent behind a sequence of requests rather than the shape of any one flow. Both web-attack families, at 0.00%, fail for the same reason.

The operative distinction is therefore between traffic that is statistically distinctive and traffic that is statistically ordinary, not between attack categories. This also explains why the model is useful despite its aggregate recall: the flows it detects are not a random subset of attacks but a specific kind.

Its ceiling has a measurable cause. On the validation set, benign flows produce a mean reconstruction error of 0.000464 and attacks 0.009814, a separation of roughly twenty-one times, which confirms that the mechanism works. But a substantial share of attack flows produce errors that fall inside the benign range, and no threshold can separate overlapping distributions. The constraint is the feature set — flow-level statistics, with no inspection of packet contents — not the tuning of the model.

One qualification applies throughout. The Autoencoder is the least stable of the three models, and its per-family percentages move substantially between runs. The pattern is stable; the individual numbers should be read as one run.

6.4 Implications of the Held-Out Attack-Family Result

Every other result in this paper shares a structural advantage: each attack family in the test set also appears in training. Production networks do not work that way. Table 7 removes that advantage for one family.

LightGBM's recall on the withheld family falls from 99.91% to 10.04%, and to 2.42% once its threshold is also chosen without the family. The second figure matters because a threshold selected on data containing the attack is itself a form of prior exposure; removing it leaves almost nothing. The model does not recognise FTP-Patator as a kind of attack. It recognises the specific patterns it was shown.

The exploratory analysis points to why. Every attack family in this capture occurs on a single day, most families target a single destination port, and destination port is LightGBM's most-used feature. FTP-Patator targets port 21 in 99.99% of its flows, so a model that has never seen an attack on that port has little reason to treat it as suspicious. This is an explanation consistent with the data rather than a demonstration; confirming it would require repeating the experiment with those features removed.

Logistic Regression lost 0.54 points. This is not evidence that the simpler model generalises better. Its false-positive rate is roughly forty times LightGBM's, so part of its coverage comes from a boundary broad enough to catch unfamiliar traffic incidentally. What the result does show is that a coarse decision rule survives an unfamiliar attack better than one fitted closely to the training families.

The Autoencoder's weights are unaffected by the exclusion, since it never trains on attacks; only its threshold changes. At its F1-tuned threshold it detects 61.08% of the withheld family, six times LightGBM's figure under the same conditions. At a threshold set from benign traffic alone — the only setting genuinely available when the attack is new, because it requires no attack labels — it detects 99.01%, at a false-positive rate of 5.06%. Neither number makes it a replacement for the supervised model. Together they establish that it covers a failure mode the supervised model cannot.

This is the finding that turns the two-model design from a preference into a measured result.

6.5 Model Selection and Operating Point

Section 5.1 reports which model performs best. This subsection sets out the criteria behind the choice, the reasons the alternatives were rejected rather than merely ranked lower, and the threshold at which the selected model should run.

The final model was selected on validation AP, with recall, false positive rate and per-family coverage examined as supporting criteria. Training time was recorded but not weighted heavily: all three models train in minutes, and the cost of operating a detector is driven by prediction speed.

For a deployed system the false positive rate is the criterion that decides feasibility rather than rank order. LightGBM raised 165 false alarms across 419,158 benign flows; the two alternatives raised 6,573 and 12,998 for the same traffic (Table 4). A single analyst can review the first figure and not the others, so the gap is the difference between a detector that would be used and one that would be switched off.

Neither alternative was rejected on aggregate performance alone. Logistic Regression is the most interpretable of the three and its recall is respectable, but it detects no Bot traffic at all (Table 5); a complete blind spot for a real attack family disqualifies a primary detector regardless of how well it can be explained. The Autoencoder misses roughly one attack in four and produces the highest alert volume, so it has no claim to the primary role — yet Section 6.4 shows it is the only model whose detection does not depend on the attack having been seen before. It is therefore retained as a second layer rather than discarded.

A caveat applies to the winner. LightGBM's most-used features are the destination port, the initial TCP receive-window sizes announced by each side of a connection, and inter-packet timing. The exploratory analysis shows that the first two partly encode which services and protocols appear in this particular capture, so the model's interpretability does not guarantee that what it learned transfers elsewhere.

The threshold is a separate decision, and one of the objectives stated in our proposal. A model outputs a score between 0 and 1, not a verdict; the threshold converts that score into an alert. Lowering it catches more attacks and raises more false alarms. The choice is a policy decision about relative cost, not a technical one, which is why we report two defensible points rather than one.

Table 8. LightGBM at two operating points on the test set.

Operating point	Threshold	Attacks missed	False alarms
Balanced (F1-optimal)	0.8467	143	165
Recall-oriented (F2-optimal)	0.3288	22	341

The threshold used in Section 5 maximises F1, which weights a missed intrusion and a false alarm equally — an assumption that does not hold in security. F2 weights recall four times as heavily. Moving to it detects 121 additional attacks for 176 additional false alarms: roughly one and a half extra alerts for each attack caught. Since a missed intrusion can cause real damage while a false alarm costs review time, and 341 alerts remain within one analyst's capacity, we recommend the recall-oriented point, with the balanced point reported for reference.

6.6 Threats to Validity

A recall of 0.9983 with 165 false alarms invites scepticism, and the checks reported in Section 5.5 were run before the results were trusted. What follows is what they establish and what they leave open.

What the checks establish. The near-zero train-test gaps rule out memorisation of the training rows. The strongest single feature separates the classes at a ROC-AUC of 0.7472, far from the value a leaked target would produce, so performance comes from combinations of features rather than from one feature encoding the answer. The protocol described in Sections 4.1 and 4.2 — fitting every data-dependent step on the training split, removing repeated flow patterns before splitting, and asserting that no test record repeats a training record — removes the most common way a score of this size becomes an artefact.

What they do not establish. All of it concerns the relationship between the splits of one capture. None of it addresses whether the capture is representative. The exploratory analysis documents structure that a model can exploit without learning anything about malicious behaviour: each attack family occurs on a single day, most families target one destination port, and the sentinel value of the initial forward window size appears almost exclusively in benign flows. A single-feature scan cannot detect a shortcut of this kind, because it is the combination of capture artefacts that carries the signal.

What follows from that. The held-out experiment in Section 6.4 is the only measurement in this study that tests past the capture's own structure, and the supervised advantage largely disappears when it does. The within-capture figures should therefore be read as an upper bound on what these models would achieve on traffic from another network, not as an estimate of it.

How the pipeline changed during development. The evaluation protocol described above was not the first one we used. An earlier version selected features before splitting the data and did not remove repeated flow patterns, so the test set both influenced the feature set and contained records the models had already seen. Both issues were identified during internal review and the pipeline was rebuilt around the protocol reported here. The supervised results stayed in the same range afterwards, which indicates they were not produced by records shared between the splits.

A note on metrics. Accuracy appears nowhere in our claims. Section 5.5 shows why: a classifier labelling every flow benign scores above 83% on this data while detecting nothing at all.

7. Limitations

The following constrain what can be concluded from this study. Each states the constraint and, where one exists, what would resolve it.

Single capture, random split. Training and test data come from the same network, the same capture period and the same attack tools, and flows from the same session appear in all three splits. Because each attack family occurs on one day, a chronological or day-based split could not evaluate every family, so a random split was used. Validation on a second capture from a different network would be required to establish how far the results transfer.

Capture-specific shortcut features. Destination Port and the `Init_Win_bytes_forward` sentinel separate much of the traffic by service and protocol rather than by malicious behaviour. Twelve of the fourteen attack families use a single destination port in essentially all their flows, while benign traffic is dominated by ports 53, 443 and 80; the sentinel value `-1` occurs in 44% of benign flows and in none of the attack flows, mostly marking DNS traffic. Both features plausibly contribute to the within-capture scores and to the collapse reported in Section 6.4. Repeating the pipeline without them would quantify their contribution.

Label-dependent filtering before the split. Patterns whose feature vectors were identical but whose labels disagreed — 698 of them — were removed using the labels themselves, since no model can separate identical inputs. The evaluated population is therefore defined in part by the target, and the reported metrics describe that filtered population rather than the raw capture.

Exact duplicates only. Flows differing by a single byte or microsecond are treated as distinct and can still fall on both sides of the split. The dataset provides no session or host identifier, so a grouped split was not possible; a distribution retaining those identifiers would resolve it.

Rare families. As Section 3 notes, several families contain very few records; in the test split Infiltration has seven, SQL Injection six and Heartbleed one. Their percentages in Table 5 reflect sample size rather than capability, and they are excluded from our detection claims. Only a dataset containing more instances of these attacks would resolve this.

Flow-level features only. No inspection of packet contents was available. This caps what any model here can separate and is the direct cause of the Autoencoder's recall ceiling discussed in Section 6.3.

Autoencoder instability. Its AP varied between roughly 0.75 and 0.90 across runs and machines, owing to random initialisation and differences in numerical libraries, even with fixed seeds; the ordering of the bottleneck-width comparison reversed between runs. Both supervised models reproduce exactly. Autoencoder figures should therefore be read as representative of one run rather than as exact.

Artefacts in the published flow features. The dataset contains physically impossible values: negative durations and rates, and overflow artefacts in rate-based fields. These were detected, quantified and handled explicitly — impossible negatives were treated as missing and imputed from the training data, while the documented -1 window sentinel was preserved as meaningful. Their underlying cause lies in the flow-extraction tool used to produce the dataset and cannot be diagnosed without the original packet captures.

8. Conclusion and Future Work

8.1 Direct Answer to the Research Question

This study addressed the core research question: how can a scalable network intrusion detection system reliably detect attacks while keeping false alarms low enough for security operations centre analysts to act upon? The empirical findings show that a class-weighted gradient-boosted decision tree ensemble (LightGBM) provides an effective solution for known attack signatures, obtaining an AP of 0.9999, a recall of 0.9983, and generating only 165 false alarms across 419,158 benign test flows. However, this operational viability is subject to specific

qualifications: performance degrades significantly on stealthy, flow-overlapping attack patterns — specifically botnet command-and-control communications — and relies heavily on having observed prior instances of specific attack signatures during training.

8.2 Principal Finding and the Two-Model Architecture

The principal finding of this work is that near-perfect supervised classification performance on known attack signatures does not transfer to unseen threats. In the held-out attack-family experiment, LightGBM's detection rate on a completely withheld attack family (FTP-Patator) collapsed from 99.91% to 10.04%. Conversely, the unsupervised Autoencoder — trained strictly on benign baseline behaviour without attack labels — detected 99.01% of the same unseen attack at a threshold set without attack labels, although at a 5.06% false-alarm rate, and 61.08% at its F1-tuned threshold. This contrast suggests that single-classifier frameworks introduce critical security gaps. Consequently, this study proposes a two-model hybrid architecture: a supervised classifier serving as a precise primary filter for known signatures, paired with an unsupervised anomaly detector operating as a safety net against novel exploits.

8.3 Prioritised Future Work

Future work to refine and extend this intrusion detection pipeline is prioritised as follows:

- Grouped and chronological validation: evaluating model stability across extended temporal windows and distinct network environments, to prevent the performance overestimation caused by same-day capture environments.
- Improved botnet command-and-control detection: incorporating stateful flow-sequence aggregation and temporal context to resolve the behavioural overlap between routine HTTP web traffic and stealthy botnet communications.
- Supervised–unsupervised hybrid ensembling: developing a formalised decision engine that unifies LightGBM probability scores with Autoencoder reconstruction errors into a single threat score.
- Removal of shortcut features: repeating the pipeline without Destination Port and the initial-window sentinel, to quantify how much of the reported performance depends on capture-specific artefacts.

References

- [1] R. Ahsan, W. Shi, and J.-P. Corriveau, "Network intrusion detection using machine learning approaches: Addressing data imbalance," *IET Cyber-Physical Systems: Theory & Applications*, vol. 7, no. 1, pp. 30–39, 2022. doi: 10.1049/cps2.12013
- [2] A. Aldweesh, A. Derhab, and A. Z. Emam, "Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues," *Knowledge-Based Systems*, vol. 189, 105124, 2020. doi: 10.1016/j.knosys.2019.105124
- [3] Z. Ahmad, A. S. Khan, C. W. Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," *Transactions on Emerging Telecommunications Technologies*, vol. 32, 2021. doi: 10.1002/ett.4150
- [4] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [5] Z. Hu, G. Liu, Y. Li, and S. Zhuang, "SAGB: Self-attention with gate and BiGRU network for intrusion detection," *Complex & Intelligent Systems*, vol. 10, pp. 8467–8479, 2024. doi: 10.1007/s40747-024-01577-y
- [6] G. King and L. Zeng, "Logistic regression in rare events data," *Political Analysis*, vol. 9, no. 2, pp. 137–163, 2001. doi: 10.1093/oxfordjournals.pan.a004868
- [7] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 3146–3154, 2017.